

Experiment 7

End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modeling.

The experiment begins with a fully connected autoencoder, progresses to a Convolutional Autoencoder (CAE), introduces image corruption for denoising, and finally develops a Variational Autoencoder (VAE). The learned latent representation is visualised, reconstruction quality is compared using quantitative and qualitative measures, and the generative capability of the VAE is explored.

The same dataset, the same train/validation/test partition and the same optimizer, batch size and epoch budget were used throughout, so that the four models can be compared as a controlled study.

2. Learning Outcomes

After completing this experiment, the following outcomes were achieved:

- the encoder–latent–decoder structure of an autoencoder was implemented and explained;
- image data were prepared for reconstruction-based learning;
- a fully connected autoencoder was implemented;
- a Convolutional Autoencoder was implemented for image reconstruction;
- controlled Gaussian and salt-and-pepper corruption were introduced and a denoising autoencoder was built;
- reconstruction was evaluated using MSE, MAE and SSIM;
- latent representations were visualised and interpreted;
- the probabilistic latent representation of a VAE was explained;
- the reparameterization trick was implemented;
- reconstruction and KL-divergence losses were computed and interpreted;
- new images were generated by sampling from the learned latent space; and
- deterministic and variational latent representations were compared.

3. Dataset and Experimental Setup

Dataset: MNIST Handwritten Digit Dataset

MNIST contains grayscale images of handwritten digits from 0 to 9, each of spatial dimension $28 \times 28 \times 1$. It suits this experiment because the images are small, the classes have visually meaningful structure, and the same format can be supplied to fully connected, convolutional and variational models. It was loaded directly through `keras.datasets.mnist.load_data()`.

Labels are never used as training targets. The image is its own target:

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}.$$

Digit labels are used in exactly one place – colouring the latent-space scatter in Section 18 – and were withheld from every model during training.

Data partition

The handout specifies 10,000 training and 2,000 test images but does not say where the validation curve required by Plot 2 should come from. A third, disjoint split was therefore carved out of the training pool:

Split	Images	Source	Purpose
Training	10,000	MNIST train[0:10000]	gradient updates
Validation	2,000	MNIST train[10000:12000]	loss curves, convergence checks
Testing	2,000	MNIST test[0:2000]	all reported metrics, evaluated once

The test images were never seen during training or used to choose any hyperparameter.

Test-set class balance

Digit	0	1	2	3	4	5	6	7	8	9
Count	175	234	219	207	217	179	178	205	192	194

Environment

Framework	TensorFlow 2.20.0 / Keras 3
Hardware	2 × NVIDIA Tesla T4 (a single device used for training)
Seed	42, re-applied before every model build
Metrics	MSE, MAE (NumPy) and SSIM (<code>skimage.metrics.structural_similarity, data_range=1.0</code>)

4. Expected Input and Output

Each input image has shape $28 \times 28 \times 1$, and for a batch of 32 images $X \in \mathbb{R}^{32 \times 28 \times 28 \times 1}$. Pixel values were converted from $[0, 255]$ to $[0, 1]$ by dividing by 255; the verified range after preprocessing was $[0.0, 1.0]$ in `float32`.

For the fully connected autoencoder the image is flattened, $28 \times 28 = 784$, so $x \in \mathbb{R}^{784}$. For the convolutional and variational models the spatial structure is retained, $x \in \mathbb{R}^{28 \times 28 \times 1}$.

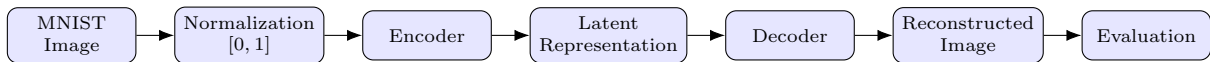
Program output:

```
Conv-format tensors (N, 28, 28, 1):
Training   : (10000, 28, 28, 1)
Validation : (2000, 28, 28, 1)
Testing    : (2000, 28, 28, 1)
Flattened (N, 784): train (10000, 784), val (2000, 784), test (2000, 784)
Pixel range: [0.0, 1.0] | dtype float32
```

The primary output is a reconstruction $\hat{x} \in \mathbb{R}^{28 \times 28 \times 1}$. Originals, reconstructions and error maps are shown in Section 8; clean, noisy and denoised triples in Section 14; and samples drawn from the latent distribution in Section 19.

Scaling to $[0, 1]$ is not cosmetic. The decoder ends in a sigmoid, whose range is exactly $(0, 1)$, and the loss is binary cross-entropy, which is only defined for targets in $[0, 1]$. Feeding raw $[0, 255]$ pixels would make the target unreachable and the loss meaningless.

5. Overall Experimental Pipeline

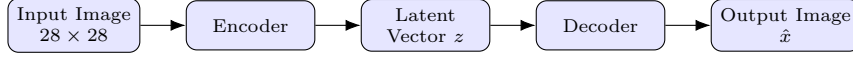


Four related studies were carried out: a fully connected autoencoder, a convolutional autoencoder, a denoising convolutional autoencoder, and a variational autoencoder.

6. Exploring Autoencoders

An autoencoder learns to reproduce its input at its output. The encoder maps the input to a lower-dimensional representation, $z = f_{\theta}(x)$, and the decoder reconstructs it, $\hat{x} = g_{\phi}(z)$. Training minimises the reconstruction error

$$L_{\text{rec}} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2.$$

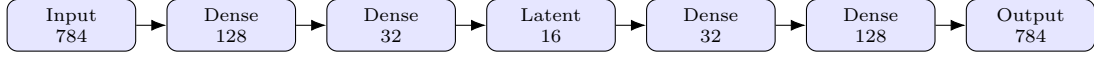


The learning signal comes entirely from the data itself, so the procedure is self-supervised. What makes the task non-trivial is the bottleneck: if z were as large as x and the layers linear, the network could learn the identity and nothing would be gained. Forcing the information through a narrow layer makes the network keep whatever is most predictive of the pixels – stroke position, thickness, slant – and discard the rest.

In this implementation binary cross-entropy was used as the training loss rather than squared error, since the pixels lie in $[0, 1]$ and the decoder ends in a sigmoid; MSE, MAE and SSIM are then reported as evaluation metrics.

7. Fully Connected Autoencoder

Each image was flattened to a vector of length 784 and passed through a symmetric encoder-decoder stack.



ReLU was used in all hidden layers, including the latent layer, and sigmoid in the output layer.

Parameter	Value
Latent dimension	16
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Epochs	20
Loss	Binary Cross-Entropy
Trainable parameters	211,040

Parameter breakdown (verified against `model.summary()`):

$$\underbrace{784 \cdot 128 + 128}_{100,480} + \underbrace{128 \cdot 32 + 32}_{4,128} + \underbrace{32 \cdot 16 + 16}_{528} + \underbrace{16 \cdot 32 + 32}_{544} + \underbrace{32 \cdot 128 + 128}_{4,224} + \underbrace{128 \cdot 784 + 784}_{101,136} = 211,040.$$

Note where the parameters live: the two layers touching the 784-pixel vector hold 201,616 of the 211,040 weights, i.e. 95.5% of the model. The compression itself is almost free; connecting to raw pixels is what is expensive. This is the first hint of why convolution wins in Section 11.

The compression ratio achieved at the bottleneck is $784/16 = 49:1$.

8. Required Analysis of the Basic Autoencoder

Plot 1: Original versus Reconstructed Images

Plot 1: Original vs Reconstruction vs Reconstruction Error (FC autoencoder, latent = 16)

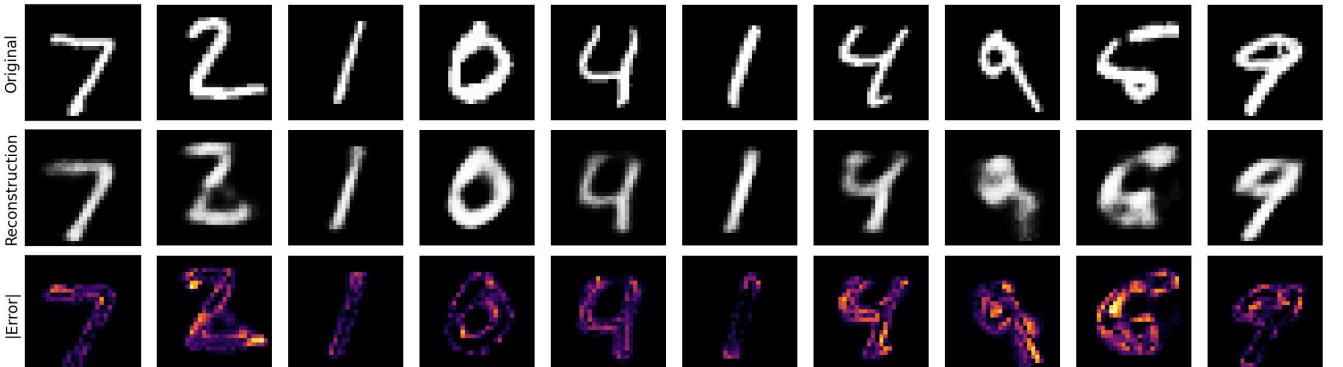


Figure 1: Plot 1: Original, reconstruction and absolute reconstruction error for ten test images (FC autoencoder, latent = 16). Brighter pixels in the bottom row mean larger error.

Per-image MSE (mean)	0.02100
Per-image MSE (minimum)	0.00142
Per-image MSE (maximum)	0.07086
Digits with the five lowest errors	1, 1, 1, 1, 1
Digits with the five highest errors	8, 8, 3, 2, 8

Comments required by the handout

- **Preservation of digit shape.** Every digit remains recognisable. Global properties – identity, position, slant, approximate stroke path – all survive the 49:1 compression, which is the main claim being tested.
- **Loss of fine details.** Stroke endpoints are rounded, small serifs disappear, and the reconstructions are systematically thinner and fainter than the originals. The error maps show that error is concentrated *along stroke boundaries*, not in the background: the model knows where the ink is but not its exact extent. Background pixels are reconstructed almost perfectly, which is unsurprising given that 88% of a typical MNIST image is background (mean ink fraction 0.1209).
- **Blurred regions.** Closed loops suffer most. The interiors of the 0 and of the loops in 8 and 9 come out grey rather than black, because the 16-dimensional code cannot specify the exact loop radius; the decoder hedges by averaging over the plausible radii it saw in training, and an average of sharp images is a blurry image.
- **Easy versus difficult samples.** The five easiest test images are all 1s and the five hardest are three 8s, a 3 and a 2. A 1 is a nearly straight stroke described by slant, length and thickness – a handful of numbers. An 8 requires two loops, their relative sizes, and the crossing point. The per-image error therefore spans a factor of 50 (0.00142 to 0.07086) across the same test set, which is why a single average MSE hides a great deal (Section 23).

Plot 2: Training and Validation Reconstruction Loss

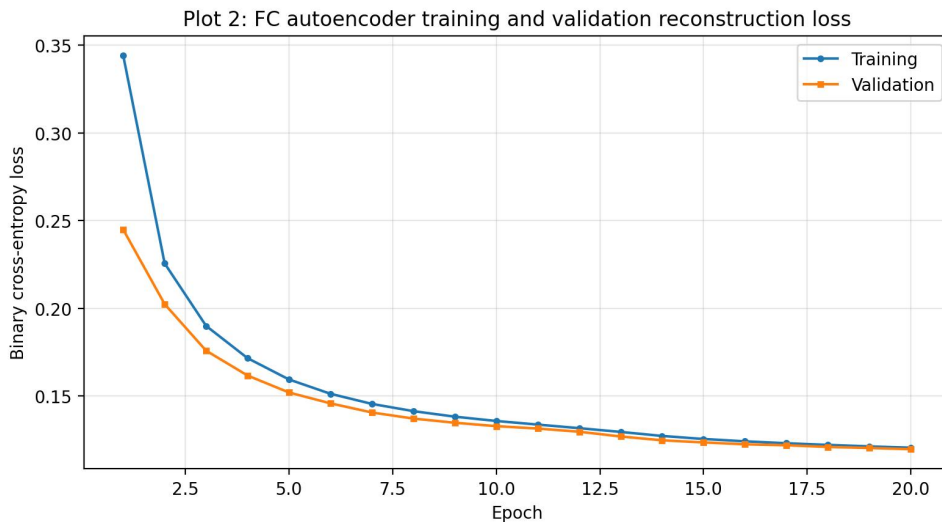


Figure 2: Plot 2: Training and validation binary cross-entropy for the fully connected autoencoder.

Final training loss	0.1204
Final validation loss	0.1196
Generalisation gap (val – train)	–0.0008
Minimum validation loss	0.1196 at epoch 20 (the last epoch)

Required inference: convergence and overfitting.

- **Does it converge?** Partially. The loss falls steeply for roughly six epochs ($0.3442 \rightarrow 0.1512$), then improves slowly and smoothly. It has *not* plateaued: validation loss is still at its minimum at the final epoch and was still decreasing by about 0.0006 per epoch. Training longer would continue to help slightly.
- **Is there overfitting?** No, and it cannot be produced here by accident. The validation curve lies marginally *below* the training curve for the whole run and the final gap is -0.0008 . Two reasons: the training loss reported by Keras is averaged over the epoch while weights are still changing, so it lags the end-of-epoch validation measurement; and with a 16-unit bottleneck the model has far too little capacity to memorise 10,000 images.
- **Why the bottleneck prevents overfitting.** The parameter count (211,040) exceeds the number of training pixels the network must fit, but every image has to pass through 16 numbers. Memorisation would require storing

per-image information, and there is nowhere to store it. The limiting factor here is representational capacity at the bottleneck, not the number of weights.

- **Conclusion.** The model is *underfitting* relative to its final achievable quality – the correct remedy is a wider latent space (confirmed in Section 27) or a better-suited architecture (confirmed in Section 11), not regularisation or early stopping.

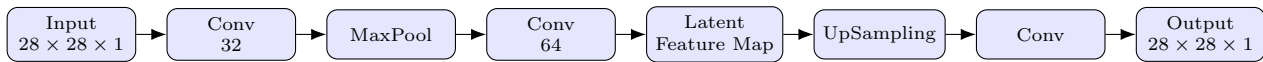
9. Reconstruction Metrics

Metrics were computed on the 2,000 untouched test images, with SSIM averaged per image at `data_range=1.0`.

Metric	Value
Test MSE	0.02100
Test MAE	0.05547
Mean SSIM	0.75181

An MSE of 0.021 corresponds to an average per-pixel deviation of $\sqrt{0.021} \approx 0.145$, i.e. about 15% of the full intensity range. MAE is much smaller (0.055) than that RMS figure, which tells us the error is not spread evenly: most pixels are nearly perfect and a minority – the stroke edges – are badly wrong. SSIM at 0.752 is the harshest of the three, because it compares local means, variances and covariances rather than pixel values, and is therefore sensitive to exactly the blur that MSE tolerates.

10. Convolutional Autoencoder



The architecture given in the handout was implemented exactly:

Layer	Output shape	Parameters
Input	(28, 28, 1)	0
Conv2D(32, 3, ReLU, same)	(28, 28, 32)	320
MaxPooling2D(2, same)	(14, 14, 32)	0
Conv2D(64, 3, ReLU, same)	(14, 14, 64)	18,496
MaxPooling2D(2, same)	(7, 7, 64)	0
Conv2D(64, 3, ReLU, same) – bottleneck	(7, 7, 64)	36,928
UpSampling2D(2)	(14, 14, 64)	0
Conv2D(32, 3, ReLU, same)	(14, 14, 32)	18,464
UpSampling2D(2)	(28, 28, 32)	0
Conv2D(1, 3, sigmoid, same)	(28, 28, 1)	289
Total trainable parameters		74,497

An important caveat about this bottleneck. The latent representation here is a $7 \times 7 \times 64$ feature map, i.e. 3136 numbers – four times *larger* than the 784-pixel input. As specified, the CAE is an overcomplete representation, not a compressive one. It is not doing the same job as the 16-dimensional FC bottleneck, and the comparison in Section 11 must be read with that in mind. A like-for-like comparison at matched latent size is run separately in Section 29 (Exercise 1), where an explicit dense bottleneck is inserted into the convolutional model.

11. Comparing Fully Connected and Convolutional Autoencoders

Both models were trained on the same 10,000 images with the same optimizer, learning rate, batch size and epoch budget.

Model	MSE	MAE	SSIM	Parameters	Time (s)
Fully Connected AE	0.02100	0.05547	0.75181	211,040	14.05
Convolutional AE	0.00241	0.01464	0.97404	74,497	23.13

The CAE reduces MSE by 88.5% and raises SSIM by 29.6% relative to the FC model, using 65% fewer parameters.

Plot 3: Fully connected vs convolutional autoencoder reconstruction

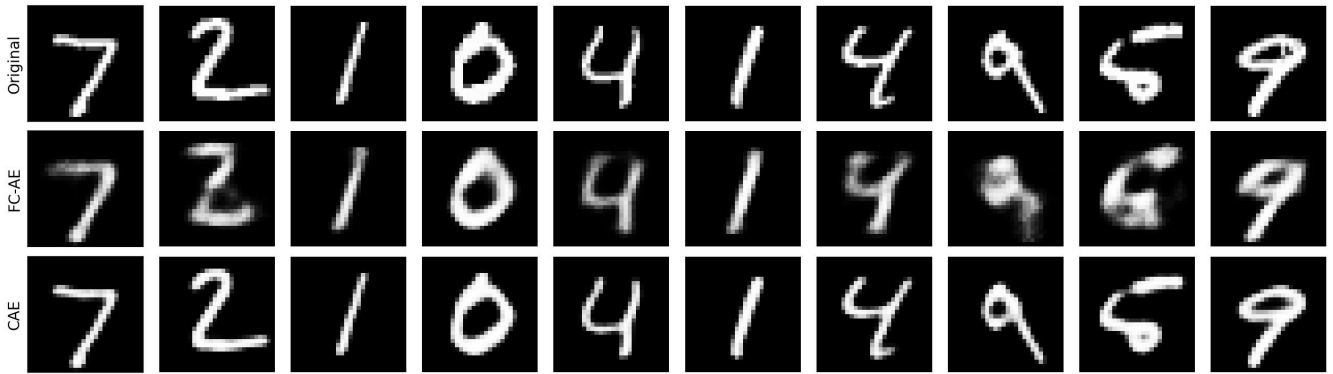


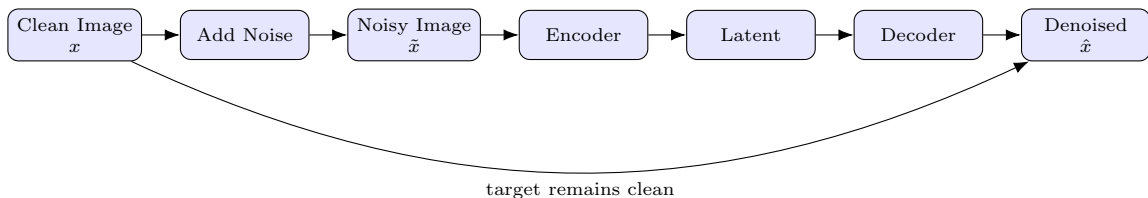
Figure 3: Plot 3: Original (top), fully connected reconstruction (middle) and convolutional reconstruction (bottom).

Required inference: how preserving spatial locality affects reconstruction.

- **What the plot shows.** The FC row is visibly blurred and thinned, with grey loop interiors and lost endpoints. The CAE row is close to pixel-identical; even the unusual samples (the 9 with a long tail, the odd 5/6-like glyph) are reproduced with their quirks intact.
- **Why locality helps.** A dense layer treats the 784 pixels as an unordered list: neighbouring pixels have no special relationship, and every spatial position needs its own weights. Convolution builds in two priors that are true of images – *locality* (a pixel is explained by its neighbourhood) and *translation equivariance* (a stroke means the same thing wherever it appears). A 3×3 kernel is reused at all 784 positions, so one learned edge detector serves the whole image, and the pooling/upsampling structure keeps the code spatially indexed, so the decoder always knows *where* a feature belongs.
- **Why the CAE also has fewer parameters.** The FC model spends 201,616 weights on the two layers that touch raw pixels. The CAE's largest layer has 36,928 weights, because kernels are shared rather than replicated per position. Better inductive bias and cheaper model at once.
- **The honest qualification.** Part of the gap is representational size, not architecture: the CAE carries 3136 latent numbers versus 16. When the convolutional model is forced through a comparable bottleneck (Section 29, Exercise 1), at $d_z = 16$ it reaches MSE 0.01327 / SSIM 0.853 against the FC model's 0.02100 / 0.752. Convolution still wins clearly – a 37% lower MSE at identical latent size – but by a smaller margin than the headline table suggests.
- **Cost.** The CAE takes 65% longer to train (23.1s versus 14.1s) despite having fewer parameters, because convolution performs far more arithmetic per parameter – each kernel is applied at every spatial position. Parameter count and compute are independent axes.

12. Denoising Autoencoder

A denoising autoencoder receives a corrupted image $\tilde{x}$ but is trained to reconstruct the clean image x :



The architecture is byte-for-byte the CAE of Section 10 (74,497 parameters); only the input changes. This isolates the effect of the training objective from the effect of the architecture.

13. Adding Image Noise

Two corruption types were used.

Gaussian noise: $\tilde{x} = \text{clip}(x + n, 0, 1)$ with $n \sim \mathcal{N}(0, \sigma^2)$ and $\sigma \in \{0.1, 0.2, 0.3\}$.

Salt-and-pepper noise: a fraction p of pixels is replaced, half set to 0 and half to 1, with $p \in \{0.05, 0.10, 0.20\}$.

Noise was generated with fixed seeds so that every model is evaluated on exactly the same corrupted images. The denoising model was trained on $\sigma = 0.2$ Gaussian noise only.

The two noise types are statistically opposite, which is visible in their metrics before any model touches them:

Corruption	MSE	MAE	SSIM
Gaussian $\sigma = 0.1$	0.00541	0.04396	0.67900
Salt-and-pepper $p = 0.05$	0.02416	0.02508	0.70854

Salt-and-pepper has $4.5\times$ the MSE of Gaussian $\sigma = 0.1$ but a *lower* MAE. Gaussian noise perturbs every pixel slightly; salt-and-pepper destroys a few pixels completely. MSE squares the error, so it is dominated by the few catastrophic pixels; MAE does not, so it reports the many small ones. This single pair of rows is the clearest argument in the experiment for reporting more than one metric.

14. Required Denoising Plots

Plot 4: Clean vs. Noisy vs. Denoised

Plot 4: Clean vs Noisy vs Denoised (Gaussian noise, sigma = 0.2)

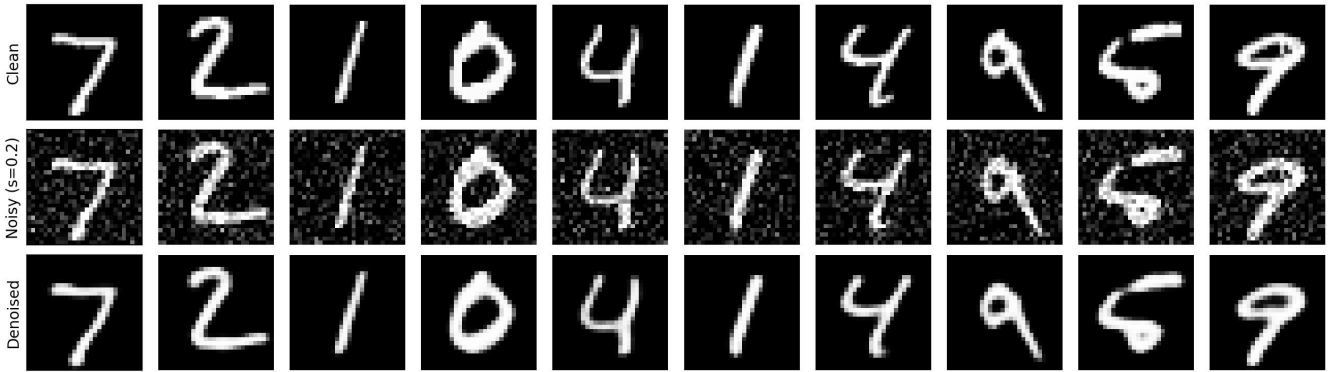


Figure 4: Plot 4: Clean, noisy ($\sigma = 0.2$) and denoised images for ten test digits.

At $\sigma = 0.2$ the background is visibly speckled and the strokes are broken up, yet the denoised row is clean, with a uniformly black background and continuous strokes. The recovered digits are slightly smoother and thicker than the originals: the model returns the most probable clean digit consistent with the noisy evidence, and that estimate is an average over plausible clean images.

Plot 5: Noise Level vs. Reconstruction Quality

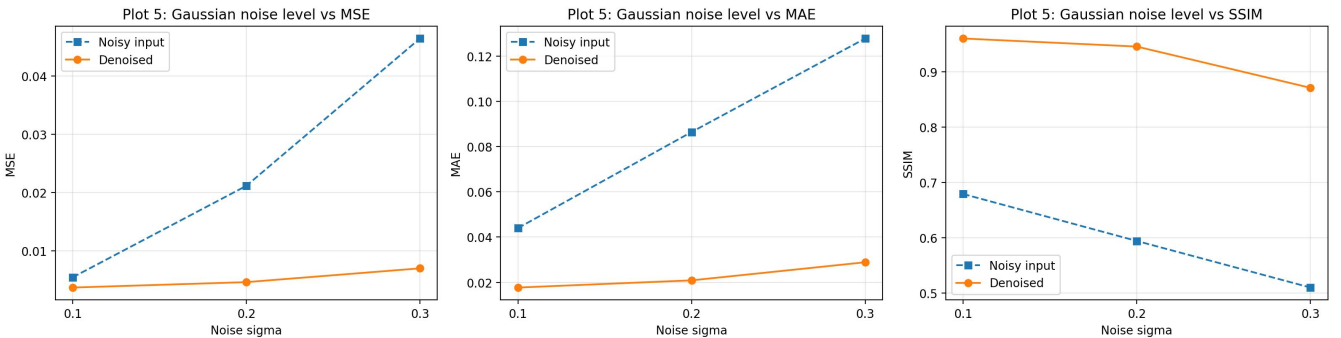


Figure 5: Plot 5: MSE, MAE and SSIM against Gaussian noise level, for the raw noisy input (do-nothing baseline) and the denoised output.

	Noisy input (baseline)			Denoised output		
Gaussian σ	MSE	MAE	SSIM	MSE	MAE	SSIM
0.1	0.00541	0.04396	0.67900	0.00366	0.01770	0.96009
0.2	0.02114	0.08640	0.59411	0.00459	0.02088	0.94549
0.3	0.04648	0.12772	0.50993	0.00696	0.02887	0.87093

	Noisy input (baseline)			Denoised output		
Salt-and-pepper p	MSE	MAE	SSIM	MSE	MAE	SSIM
0.05	0.02416	0.02508	0.70854	0.00472	0.02067	0.93048
0.10	0.04827	0.05012	0.57784	0.00672	0.02640	0.87687
0.20	0.09652	0.10020	0.44577	0.01287	0.04281	0.73986

(The salt-and-pepper rows use the same model, which never saw that corruption during training.)

Required inference: effect of corruption level and recovery of structure.

- **What the plot shows.** Two curves per metric: what the corruption costs if nothing is done, and what remains after denoising.
- **Trend.** Input quality degrades quickly and roughly quadratically in σ – MSE $0.0054 \rightarrow 0.0211 \rightarrow 0.0465$, tripling then doubling. Output quality degrades slowly and almost linearly – $0.00366 \rightarrow 0.00459 \rightarrow 0.00696$. Over the same range SSIM falls from 0.679 to 0.510 at the input but only from 0.960 to 0.871 at the output. At $\sigma = 0.3$ the model removes 85% of the squared error.
- **Why.** The digit is low-dimensional and highly structured; the noise is independent across pixels and structureless. An encoder that has learned the manifold of digits simply cannot represent i.i.d. speckle – the bottleneck has no code for it – so projecting onto that manifold discards noise almost for free. What degrades with σ is not the model’s ability to remove noise but the amount of genuine evidence left: at high σ thin strokes are buried, and the model must guess.
- **Is the major structure recovered?** Yes, comfortably. SSIM stays above 0.87 at every Gaussian level tested, against 0.51–0.68 for the untreated input. Plot 4 confirms it visually: identity, position and stroke topology are all preserved. What is lost is fine detail – exact stroke width and endpoint sharpness.
- **A cost worth naming.** At $\sigma = 0.1$ the denoiser reaches MSE 0.00366, which is *worse* than the plain CAE’s 0.00241 on clean input. The denoising objective buys robustness and pays with a little fidelity: the model is trained to smooth, so it smooths even when there is nothing to remove.
- **Cross-noise behaviour.** Applied to salt-and-pepper, which it never saw, the same model still lifts SSIM from 0.578 to 0.877 at $p = 0.10$. Denoising generalises partially across corruption types, but not fully – quantified in Section 29, Exercise 2.

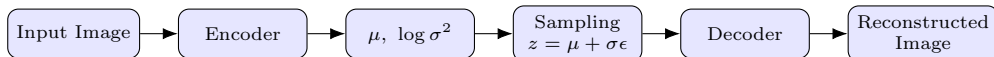
15. Variational Autoencoder

A conventional autoencoder maps an input to a deterministic latent vector $z = f(x)$. A VAE instead learns a distribution,

$$q_\phi(z|x) = \mathcal{N}(\mu(x), \text{diag}(\sigma^2(x))),$$

with the encoder producing μ and $\log \sigma^2$. A latent vector is drawn using the reparameterization trick,

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$



The trick exists for one reason: $z \sim \mathcal{N}(\mu, \sigma^2)$ is a random node, and gradients cannot flow through sampling. Writing z as a deterministic function of μ , σ and an *external* random draw ϵ moves the randomness off the computational path, so $\partial z / \partial \mu = 1$ and $\partial z / \partial \sigma = \epsilon$ are both well defined and the encoder can be trained by ordinary backpropagation.

16. VAE Loss Function

The objective has two components. The reconstruction term,

$$L_{\text{rec}} = -\mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)],$$

implemented as binary cross-entropy *summed over the 784 pixels* and averaged over the batch. The KL term,

$$D_{\text{KL}}(q_\phi(z|x) \| p(z)) = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2),$$

with prior $p(z) = \mathcal{N}(0, I)$. The total loss is $L_{\text{VAE}} = L_{\text{rec}} + \beta L_{\text{KL}}$, with $\beta = 1$ for the main model.

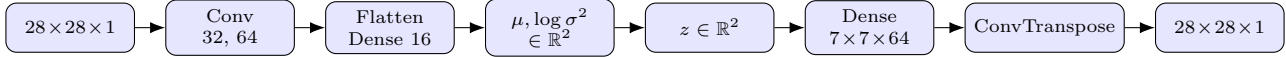
The two terms pull in opposite directions, and that tension is the whole point. Reconstruction wants each image to get its own distinct, confident code; KL wants every posterior to look like $\mathcal{N}(0, I)$. Without KL the encoder would

scatter codes anywhere it liked and the space between them would decode to noise – generation would be impossible. Without reconstruction every posterior would collapse onto the prior and all images would decode identically.

Summing rather than averaging the cross-entropy over pixels matters: it fixes the relative weight of the two terms at roughly 155 : 6 in the converged model. Had the reconstruction term been averaged over pixels it would have been 784 times smaller, the KL would have dominated, and the latent space would have collapsed.

17. VAE Implementation

A two-dimensional latent space was used so that it can be plotted directly.



Component	Structure	Parameters
Encoder	Conv(32, s2) → Conv(64, s2) → Flatten → Dense(16) → $\mu, \log \sigma^2$	69,076
Decoder	Dense(7·7·64) → Reshape → ConvT(64, s2) → ConvT(32, s2) → Conv(1, sigmoid)	65,089
Total		134,165

Training used Adam at 10^{-3} , batch size 128 and 30 epochs. Note that $\log \sigma^2$ is predicted rather than σ : it is unconstrained in sign, so no activation is needed to keep the variance positive, and $\sigma = \exp(\frac{1}{2} \log \sigma^2)$ is numerically stable.

18. Exploring the VAE Latent Space

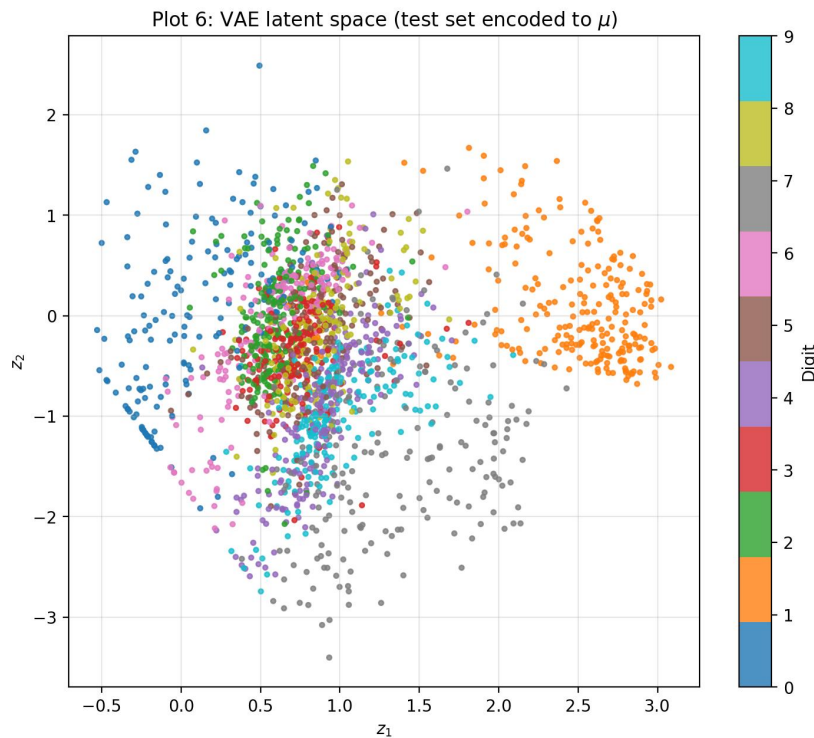


Figure 6: Plot 6: The 2-D latent space, with the 2,000 test images encoded to their means μ and coloured by digit label.

Digit	z_1	z_2	Mean radius	Digit	z_1	z_2	Mean radius
0	0.122	0.045	0.814	5	0.845	-0.152	0.542
1	2.432	0.083	0.622	6	0.600	-0.088	0.659
2	0.608	-0.018	0.490	7	1.334	-1.354	0.877
3	0.739	-0.329	0.400	8	0.854	-0.130	0.515
4	0.903	-0.922	0.634	9	0.983	-0.966	0.525

Global latent statistics: $\text{mean}(\mu) = 0.295$, $\text{std}(\mu) = 1.016$, $\text{mean}(\sigma) = 0.071$.

Required inference: do same-digit samples occupy similar regions, and do different digits overlap?

- **Partially, and unevenly.** Three groups are genuinely separated. Digit 1 forms a tight, isolated island on the right ($z_1 \approx 2.43$, more than 1.4 units from any other centroid, with a spread of only 0.62). Digit 7 occupies the lower region (1.334, -1.354), and 4 and 9 share the lower-centre (0.903, -0.922) and (0.983, -0.966). Digit 0 sits at the far left of the main mass.
- **Everything else is a single overlapping blob.** The centroids of 2, 3, 5, 6 and 8 all lie within a circle of radius roughly 0.15 around (0.73, -0.14), while their individual spreads are 0.40–0.66 – the clusters are three to four times wider than the distances between their centres. These digits are not separated at all.
- **Why.** Two dimensions is a severe constraint. Ten digit classes, each with its own variation in slant, thickness and style, cannot be laid out in a plane without collisions. The classes that *do* separate are the ones with the most distinctive global shape: a 1 is a single stroke with very little ink, while 3, 5 and 8 share curved strokes in the same places and are mutually confusable at low resolution.
- **The labels were never used.** The structure that exists is unsupervised – it emerges because visually similar images must decode to similar pixels, so the encoder is pushed to place them nearby.
- **Two quantitative warnings.** First, $\text{mean}(\sigma) = 0.071$ is tiny next to $\text{std}(\mu) = 1.016$: each posterior is a pinprick compared with the spread of the codes, so the encoder is behaving almost deterministically and the posteriors barely overlap, leaving gaps between them. Second, the aggregate posterior is not centred on the prior – $\text{mean}(\mu) = 0.295$, and the mass sits around $z_1 \in [0.3, 1.2]$ rather than symmetrically about the origin. Both facts predict the generation problem seen in Section 19.

19. Generating New Images with the VAE

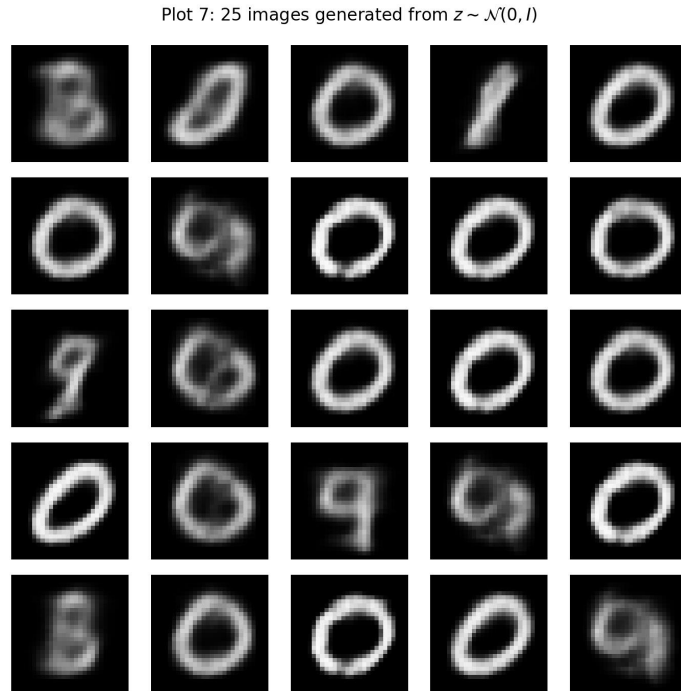


Figure 7: Plot 7: 25 images decoded from latent vectors drawn from $\mathcal{N}(0, I)$.

Mean pixel intensity: 0.1682 for the generated samples versus 0.1209 for real test images, i.e. the generated digits carry about 39% more ink.

Required assessment

- **Do they resemble handwritten digits?** Broadly yes. Every sample is a connected, centred, stroke-like white shape on a black background with roughly the right ink budget. No sample is pure noise, which already proves the decoder has learned the digit manifold rather than a lookup table.
- **Are digit shapes recognisable?** Some clearly – several 0s, a 1, a few 9s and 8-like forms. Many are ambiguous glyphs that sit between classes.
- **Is there visible variation?** Far less than in real MNIST. The grid is dominated by ring shapes: roughly 15 of the 25 samples are 0-like, and the 100-sample grid in Section 29 shows the same imbalance. Classes 2, 3, 4, 5, 6 and 7 are essentially absent.
- **Why the diversity is poor – the prior-versus-posterior mismatch.** Sampling is done from $\mathcal{N}(0, I)$, but the encoder placed the data elsewhere. The origin is closest to the digit-0 centroid (0.122, 0.045), so a large share of prior samples lands in 0 territory. Digit 1’s centroid is at $z_1 = 2.43$, which is 2.4σ out under the prior – reachable,

but rarely. The KL term is supposed to prevent exactly this drift; at 6.13 nats it is clearly not strong enough to align the aggregate posterior with the prior in two dimensions.

- **Why some samples are unrealistic.** With $\text{mean}(\sigma) = 0.071$ the posteriors are near-points, so large regions of the plane were never visited during training. A prior sample landing in such a hole is decoded by a function that was never supervised there, and the output is a plausible-looking but meaningless blend. The extra ink (0.168 versus 0.121) is the same averaging effect: an uncertain decoder hedges by drawing a thicker, fuzzier stroke.
- **Conclusion.** The VAE is genuinely generative – it produces new images from noise, which no model in Sections 7–14 can do – but at $d_z = 2$ the generated distribution is a poor match for the data distribution. Section 29 (Exercise 6) shows the direction of the fix: lowering β to 0.5 raises KL to 7.13 and improves reconstruction, while $\beta = 4$ crushes KL to 3.45 and makes everything blurrier.

20. Latent Space Interpolation

Two test images were encoded to z_A (a 0) and z_B (a 1), and $z(\alpha) = (1-\alpha)z_A + \alpha z_B$ was decoded for $\alpha = 0, 0.1, \dots, 1$.

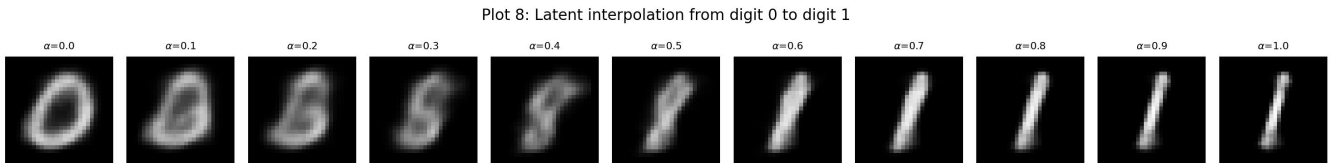


Figure 8: Plot 8: Latent interpolation from a 0 to a 1 in eleven steps.

Pixel-space MSE between consecutive decoded frames:

Step (α)	0→.1	.1→.2	.2→.3	.3→.4	.4→.5	.5→.6	.6→.7	.7→.8	.8→.9	.9→1
MSE	0.01180	0.00684	0.00907	0.00671	0.00568	0.00453	0.00287	0.00294	0.00156	0.00089

Ratio of largest to smallest step: **13.24**.

Required inference: does the decoded image change smoothly?

- **Smooth in content, uneven in pace.** There is no point at which the image jumps discontinuously; the 0 gradually loses its loop, passes through 6- and 5-like intermediates, thins into a slanted stroke and becomes a 1. Every intermediate is a plausible glyph, not a superposition of two digits or a smear – which is the property a well-behaved latent space is supposed to have, and which a plain autoencoder does not guarantee.
- **But the pacing is far from uniform.** The first step changes the image 13 times more than the last. The path starts in the crowded central region, where a small movement in z crosses several class territories, and ends in the sparse, homogeneous area occupied by 1s, where large movements barely change the output. The latent metric is not uniform: distance in z does not correspond to a constant amount of visual change.
- **Why this works at all.** The KL term forces posteriors toward a single connected blob of prior mass instead of letting them drift into isolated islands, so a straight line between two codes stays inside the region the decoder was trained on. A deterministic autoencoder has no such pressure, and its interpolations commonly pass through dead zones that decode to nothing.
- **Interpolating between class centroids** (Section 29, Exercise 8) gives the same picture with less noise: $0 \rightarrow 8$ passes through 6-like and B-like shapes, with the loop closing progressively.

21. Reconstruction Comparison Across Models

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.02100	0.05547	0.75181	211,040
Convolutional AE	0.00241	0.01464	0.97404	74,497
Denoising CAE (tested at $\sigma = 0.2$)	0.00459	0.02088	0.94549	74,497

The VAE is reported separately, since its objective also contains the KL term. Losses are per image, with the reconstruction term summed over the 784 pixels.

VAE Metric	Value
Reconstruction Loss	155.179
KL Loss	6.133
Total Loss	161.312
Test MSE	0.04611
Test MAE	0.10778
Mean SSIM	0.44859

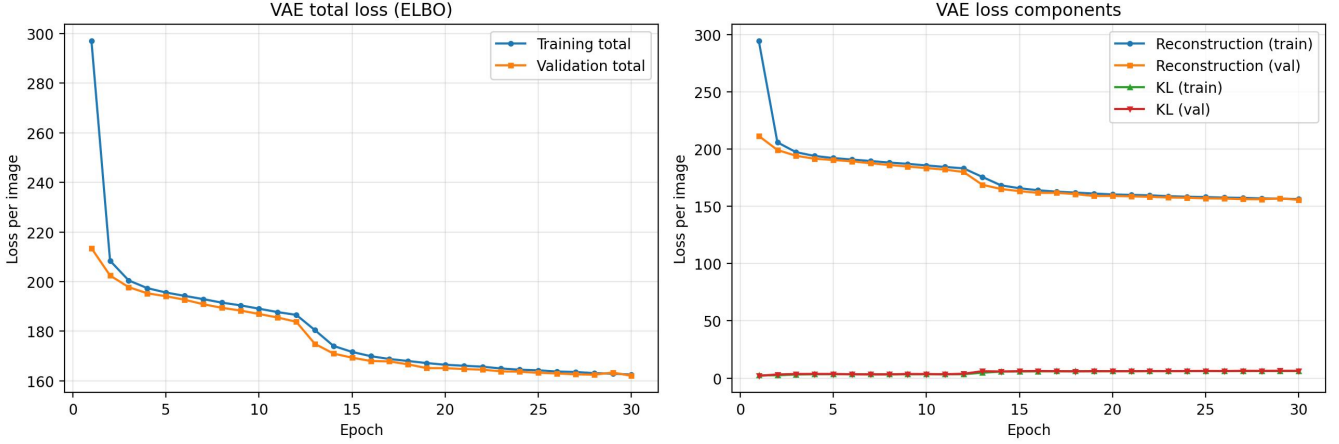


Figure 9: VAE total loss (left) and its reconstruction and KL components (right), training and validation.

Inference – VAE convergence. The total loss falls from 297.0 to 162.5 over 30 epochs, with validation tracking training closely (162.0 versus 162.5 at the end) – no overfitting. The interesting feature is the step near epoch 13: KL jumps from 3.36 to 4.69 and then to 5.63, while reconstruction loss drops sharply from 183.2 to 168.5. The model had been using effectively one latent direction and at that point started using the second. KL rising is not a failure here – it is the price paid for a more informative code, and the total loss falls because reconstruction improves by more than KL costs.

Inference – is the VAE simply a bad reconstructor? No. Its SSIM of 0.449 must be compared with the right baseline. The deterministic FC autoencoder at the *same* latent size, $d_z = 2$, scores SSIM 0.4386 and MSE 0.04716 (Section 27) – very slightly worse than the VAE’s 0.4486 and 0.04611. Almost the entire gap between the VAE and the CAE is the two-dimensional bottleneck, not the probabilistic formulation. The VAE pays only a small reconstruction premium for the KL constraint, and gets the ability to generate in return.

22. Index of Required Plots

#	Content	Figure file
1	Original versus reconstructed image	plot1_fc_reconstruction.jpg
2	Training/validation loss, basic autoencoder	plot2_fc_loss_curves.jpg
3	FC AE versus convolutional AE reconstruction	plot3_fc_vs_cae.jpg
4	Clean versus noisy versus denoised	plot4_clean_noisy_denoised.jpg
5	Noise level versus MSE / MAE / SSIM	plot5_noise_level_vs_quality.jpg
6	VAE latent-space visualisation	plot6_vae_latent_space.jpg
7	Randomly generated VAE images	plot7_vae_generated.jpg
8	Latent-space interpolation	plot8_latent_interpolation.jpg
9	Training/validation reconstruction loss for the VAE	plot_vae_loss_curves.jpg
10	Reconstruction-error distribution on the test set	plot9_error_distribution.jpg
–	Latent dimension versus reconstruction MSE (§27)	plot10_latent_dimension.jpg
–	Five highest-error test images (§24)	plot_high_error_samples.jpg
–	Salt-and-pepper denoising, matched vs mismatched (§29)	plot_extra_salt_pepper.jpg
–	UpSampling2D versus Conv2DTranspose decoder (§29)	plot_extra_decoder_comparison.jpg
–	100 generated samples (§29)	plot_extra_generated_100.jpg
–	Interpolation between class centroids (§29)	plot_extra_centroid_interpolation.jpg

Every plot carrying several quantitative measures uses separate axes rather than mixing incompatible scales: Plot 5 is drawn as three panels, and Plot 10 as two.

23. Reconstruction Error Distribution

The per-image error

$$e_i = \frac{1}{D} \sum_{j=1}^D (x_{ij} - \hat{x}_{ij})^2$$

was computed for all 2,000 test images.

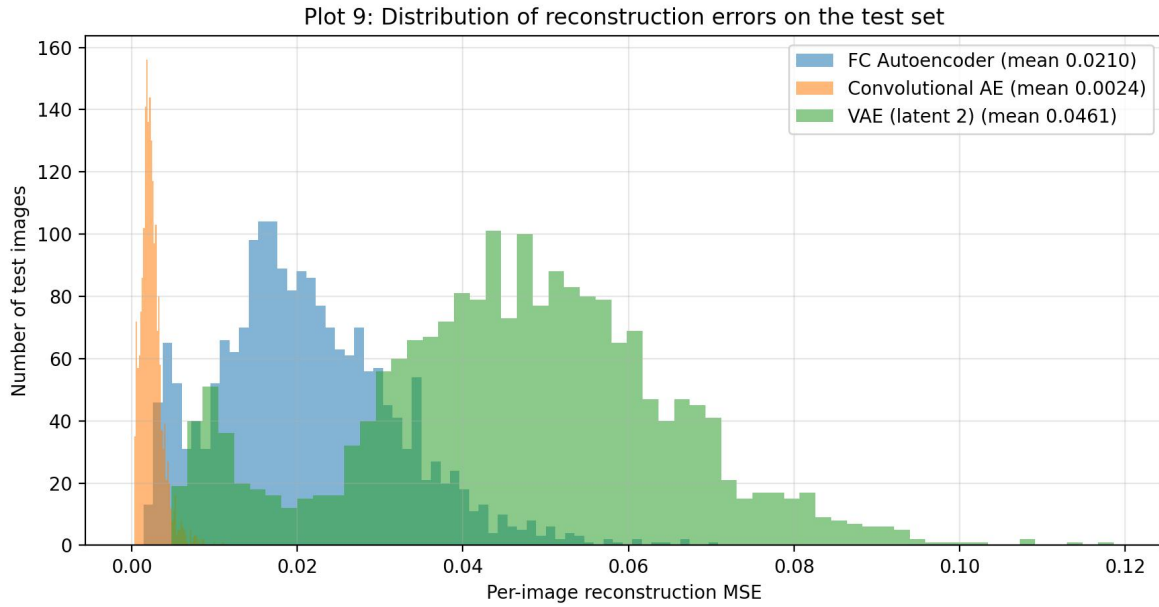


Figure 10: Plot 9: Distribution of per-image reconstruction MSE on the test set for the three reconstruction models.

Model	Mean	Median	90th percentile	Maximum	Skew
FC Autoencoder	0.02100	0.01987	0.03491	0.07086	+0.65
Convolutional AE	0.00241	0.00222	0.00397	0.01117	+1.46
VAE (latent 2)	0.04611	0.04690	0.06916	0.11862	−0.04

Required observations

- **Typical range.** The CAE occupies 0.001–0.004 for most images, the FC model 0.010–0.035, and the VAE 0.030–0.070. The three distributions barely overlap, so the ranking is not an artefact of a few outliers – it holds image by image.
- **Concentrated or spread out?** The CAE is by far the most concentrated in absolute terms (90th percentile 0.00397, only $1.6\times$ the median) and also the most right-skewed (+1.46): almost every image is reconstructed well, and a small tail of unusual digits is not. The FC model is wider and mildly skewed. The VAE is broad and nearly symmetric (−0.04): it reconstructs *everything* imprecisely rather than failing on a hard minority, exactly what a 2-D bottleneck predicts.
- **What this says about the mean.** For the CAE the mean (0.00241) sits above the median (0.00222) because of the tail, so the average slightly understates typical performance. Quoting a single mean MSE hides a factor of five between the median and the worst image.
- **High-error samples.** The upper tail is examined in Section 24.

24. Required High-Error Image Analysis

The five test images with the largest reconstruction error under the convolutional autoencoder:

Sample	Test index	True digit	Per-image MSE	Ink fraction
1	1526	0	0.01117	0.2002
2	895	0	0.01004	0.1736
3	1125	8	0.00866	0.1259
4	431	8	0.00865	0.1026
5	1320	8	0.00859	0.0800

Mean ink fraction over the whole test set: 0.1209.

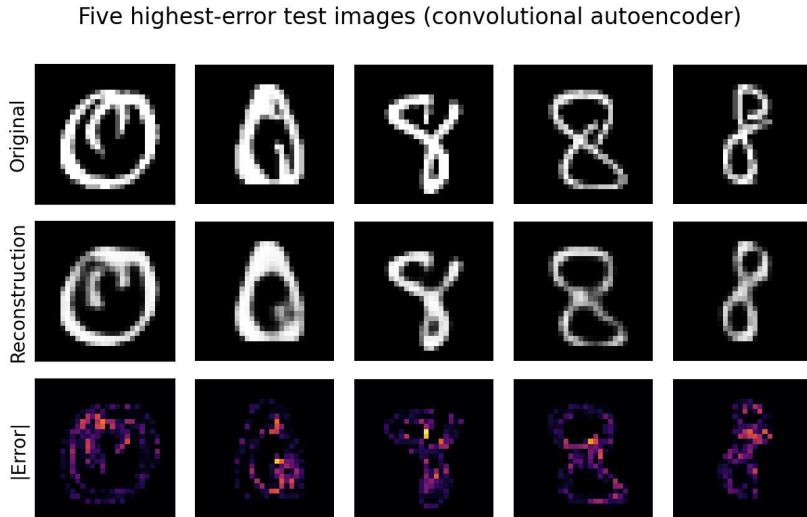


Figure 11: The five highest-error test images: original, reconstruction and absolute error.

Why these images are difficult

- **They are atypical within their own class.** The two 0s are not clean ovals – one has an extra internal stroke crossing the loop, the other a narrow, squashed shape with a hooked top. The three 8s are all unusual: one is drawn as a Y-like form with an open lower loop, one has a wide flat lower loop, one is a thin, elongated figure. The encoder compresses toward the average member of each class, and the further an image sits from that average, the more information is lost.
- **Two of the five carry unusually much ink** (0.200 and 0.174 versus a dataset mean of 0.121). More ink means more non-background pixels that can be wrong, and squared error scales with the number of mismatched pixels, so heavy digits start at a disadvantage under MSE.
- **But ink alone does not explain it**, and this is worth noting: sample 5 has an ink fraction of 0.080, well *below* average, and still ranks fifth. Its difficulty is structural – a thin, unusually shaped 8 whose two loops nearly merge.
- **Classes 0 and 8 dominate the tail** because both depend on closed loops. A loop is exactly the feature whose interior the decoder blurs first, and a small error in loop radius puts a ring of wrong pixels around the whole contour. The error maps confirm this: the brightest error pixels trace the loop boundaries.
- **Practical reading.** Reconstruction error is a usable out-of-distribution score. The images the model reconstructs worst are the ones least like its training distribution, which is precisely how autoencoders are used for anomaly detection.

25. Consolidated Results

Model	MSE	MAE	SSIM	Parameters	Time (s)
FC Autoencoder	0.02100	0.05547	0.75181	211,040	14.05
Convolutional AE	0.00241	0.01464	0.97404	74,497	23.13
Denoising CAE	0.00459	0.02088	0.94549	74,497	20.06
VAE (latent 2)	0.04611	0.10778	0.44859	134,165	33.92

The four rows are not four attempts at one task, and ranking them by MSE alone would be misleading. The CAE reconstructs clean images from a 3136-value overcomplete code. The denoising CAE is scored on *corrupted* input, so its 0.00459 is measured under a handicap the others never face. The VAE is scored through a 2-dimensional probabilistic bottleneck and is the only model of the four that can generate a new image. Each number should be read against what that model was asked to do.

26. Required Inferences (Summary)

1. **Effect of latent dimension on reconstruction.** MSE falls $0.04716 \rightarrow 0.03040 \rightarrow 0.02100 \rightarrow 0.01997$ and SSIM rises $0.439 \rightarrow 0.651 \rightarrow 0.752 \rightarrow 0.760$ for $d_z = 2, 8, 16, 32$. The gain is steep up to 16 and nearly flat from 16 to 32 (+0.008 SSIM for double the code), so 16 is the knee for this architecture.
2. **Fully connected versus convolutional reconstruction.** The CAE cuts MSE by 88.5% and raises SSIM by 29.6% with 65% fewer parameters. Weight sharing and spatial locality, not extra capacity, are responsible; at a matched $d_z = 16$ bottleneck the convolutional model still leads by 37% lower MSE.
3. **Effect of Gaussian noise level.** Input MSE grows roughly quadratically ($0.0054 \rightarrow 0.0211 \rightarrow 0.0465$) while the denoised output grows slowly ($0.00366 \rightarrow 0.00459 \rightarrow 0.00696$). At $\sigma = 0.3$ the model removes 85% of the squared error.
4. **Ability of the denoising autoencoder to recover structure.** Recovered SSIM stays above 0.87 across all Gaussian levels versus 0.51–0.68 untreated. Identity, position and topology survive; stroke width and endpoint sharpness do not. Denoising costs a little clean-image fidelity: 0.00366 at $\sigma = 0.1$ against the plain CAE’s 0.00241.
5. **Characteristics of the VAE latent distribution.** Converged KL is 6.13 nats, $\text{std}(\mu) = 1.016$ and $\text{mean}(\sigma) = 0.071$. The posteriors are much narrower than their spread, so the encoder is nearly deterministic, and the aggregate posterior is offset from the prior ($\text{mean}(\mu) = 0.295$). Both facts limit generation quality.
6. **Structure of the two-dimensional latent space.** Digit 1 is cleanly isolated at $z_1 = 2.43$; 7, and the pair 4/9, occupy distinct lower regions; 2, 3, 5, 6 and 8 collapse into one blob whose centroids are separated by less than a quarter of their own spread. Two dimensions cannot hold ten classes apart.
7. **Quality and diversity of generated samples.** Every sample is digit-like, but about 60% are 0-like rings and several classes never appear. Diversity is limited by the prior–posterior mismatch, and the excess ink (0.168 versus 0.121) reflects decoder hedging in unvisited regions.
8. **Smoothness of latent interpolation.** No discontinuities and no ghost superpositions – every intermediate is a plausible glyph – but the pacing is uneven, with the largest step 13.24 times the smallest. The space is continuous but not metrically uniform.
9. **Relationship between reconstruction error and visual quality.** Error tracks visual quality within a model but not across metrics: SSIM is much harsher on blur than MSE (FC AE: MSE 0.021 looks acceptable, SSIM 0.752 does not), and salt-and-pepper corruption has $4.5\times$ the MSE of Gaussian $\sigma = 0.1$ at a *lower* MAE. The high-error images are atypical exemplars, mostly loop-heavy 0s and 8s.
10. **Deterministic versus probabilistic latent representations.** A deterministic code is a point, is optimised only for reconstruction, and wins on fidelity (CAE SSIM 0.974). A probabilistic code is a distribution, is regularised toward $\mathcal{N}(0, I)$, and supports sampling and interpolation. The measured cost of that regularisation is small: at $d_z = 2$ the VAE (SSIM 0.4486) actually edges out the deterministic FC autoencoder (0.4386); what it buys is the ability to generate.

27. Additional Latent-Dimension Study

The basic autoencoder was retrained with $d_z \in \{2, 8, 16, 32\}$, everything else unchanged.

Latent Dimension	MSE	MAE	SSIM	Parameters	Time (s)
2	0.04716	0.11076	0.43855	210,130	13.28
8	0.03040	0.07533	0.65050	210,520	12.55
16	0.02100	0.05547	0.75181	211,040	14.05
32	0.01997	0.05420	0.75964	212,080	11.99

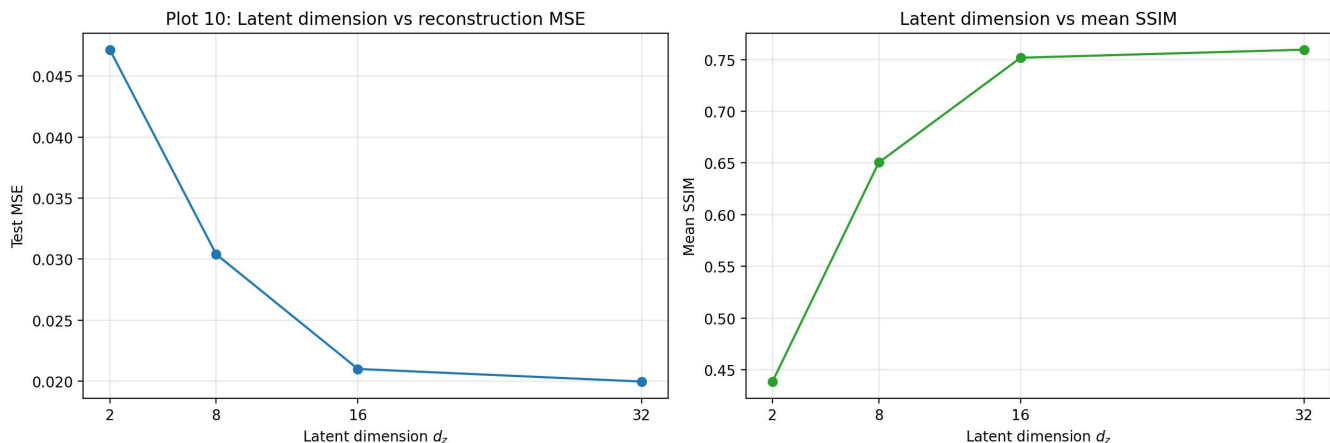


Figure 12: Plot 10: Latent dimension versus test MSE (left) and mean SSIM (right).

The compression–quality trade-off

- **Diminishing returns are sharp.** Going $2 \rightarrow 8$ buys 0.212 SSIM; $8 \rightarrow 16$ buys 0.101; $16 \rightarrow 32$ buys 0.008. Doubling the code from 16 to 32 improves MSE by 5% while halving the compression ratio from 49:1 to 24.5:1.
- **Why the curve saturates.** MNIST digits have an intrinsic dimensionality well below 32 – identity plus a handful of style factors. Once the code is wide enough to carry those factors, extra dimensions have nothing left to encode, and the remaining error comes from decoder capacity and the blur inherent in minimising a pixel-wise loss, neither of which more latent units fixes.
- **Cost is not the parameter count.** Total parameters barely move (210,130 to 212,080, under 1%) because the bottleneck layers are tiny next to the 784-wide input and output layers. Training time is essentially flat and dominated by run-to-run variation. The real cost of a larger d_z is the loss of compression and of the regularisation the bottleneck provides – not compute.
- **The $d_z = 2$ point is worth a caveat.** ReLU on the latent layer, as the handout specifies, clips negative activations, so a 2-unit ReLU code lives in the positive quadrant only and dead units are catastrophic. Part of the poor SSIM at $d_z = 2$ (0.4386) is this activation choice rather than dimensionality alone. The VAE, whose 2-D latent is linear, reaches a marginally better 0.4486 with the same code size.
- **Practical conclusion.** $d_z = 16$ is the right operating point here: it captures essentially all the quality available from this architecture at twice the compression of $d_z = 32$.

28. Discussion Questions

1. **What is an autoencoder?** A neural network trained to reproduce its own input through a constrained intermediate representation. It has two halves – an encoder $z = f_\theta(x)$ and a decoder $\hat{x} = g_\phi(z)$ – trained jointly to minimise reconstruction error. No labels are needed, so it is self-supervised: the supervision signal is the input itself.
2. **What is the purpose of the encoder?** To compress the input into a representation that retains what is needed to reconstruct it. It performs lossy, learned dimensionality reduction – 784 $\rightarrow$ 16 here, a 49:1 ratio – and the loss is chosen by training rather than by hand, so the discarded information is whatever mattered least for the pixels.
3. **What is the purpose of the decoder?** To invert the encoding: map a latent vector back to the input space. It defines what each latent code *means* as an image, which is why it can be used on its own as a generator (Section 19) – feed it any z and it produces an image, whether or not that z came from a real input.
4. **What is a latent representation?** The internal code z at the bottleneck: a compact vector of learned features. In the FC model it is 16 numbers; in the VAE it is two numbers plus their uncertainties. Its coordinates are not interpretable individually, but distance in that space is meaningful – similar images encode to nearby codes, which is what Plot 6 shows.
5. **Why is the latent representation usually smaller than the input?** To force generalisation. An unconstrained autoencoder with a code as wide as its input can learn the identity map and learn nothing about the data. Making the code narrower creates an information bottleneck: the network must decide what to keep, and to do that well it has to discover the structure the data actually has.
6. **What happens when the latent dimension is too small?** Reconstruction degrades severely and systematically. At $d_z = 2$ the model scored MSE 0.04716 and SSIM 0.43855 against 0.02100 / 0.75181 at $d_z = 16$. The outputs keep only class-level identity and coarse position; style, stroke width and individual quirks are all averaged away, so different inputs of the same class start producing near-identical outputs.
7. **What happens when the latent dimension is increased?** Reconstruction improves with diminishing

returns, then stops improving: SSIM $0.439 \rightarrow 0.651 \rightarrow 0.752 \rightarrow 0.760$ for 2, 8, 16, 32. Past the intrinsic dimensionality of the data, extra units add compute and reduce compression without adding quality, and in the limit the model drifts toward copying rather than learning structure.

8. **Why can a convolutional autoencoder preserve spatial information more effectively?** Because it never discards the spatial layout. Dense layers flatten the image, so pixel 27 and pixel 28 – adjacent in the image – become unrelated inputs, and every location needs separate weights. Convolution keeps the 2-D grid, applies the same local kernel everywhere (weight sharing plus translation equivariance), and keeps the code itself spatially indexed, so the decoder knows where each feature belongs. Measured effect: 88.5% lower MSE with 65% fewer parameters.
9. **Differentiate an autoencoder and a denoising autoencoder.** Only the input differs. A standard autoencoder maps $x \rightarrow x$; a denoising autoencoder maps $\tilde{x} \rightarrow x$ with $\tilde{x}$ a corrupted copy. That change turns copying into a genuinely harder task – the model must infer what was destroyed – which forces it to learn the data manifold instead of a shortcut. Here both used the identical 74,497-parameter architecture.
10. **Why is the clean image used as the target in denoising?** Because the target defines the task. Using the noisy image as target would teach the model to reproduce noise. Making the clean image the target forces a mapping from corrupted space onto the manifold of clean digits, which is exactly denoising. A useful by-product: noise cannot be memorised, since it is different in every sample, so this objective is inherently regularising.
11. **What happens when the noise level is increased?** Input quality collapses quickly (MSE $0.0054 \rightarrow 0.0465$ from $\sigma = 0.1$ to 0.3) while output quality degrades gently ($0.00366 \rightarrow 0.00696$, SSIM $0.960 \rightarrow 0.871$). The model keeps working because noise is unstructured and the digit manifold is low-dimensional, but past some level the genuine evidence in thin strokes is gone and the reconstruction becomes a prior-driven guess.
12. **Why are MSE, MAE and SSIM useful for image reconstruction?** They fail differently, and this experiment demonstrates it directly. MSE squares errors, so it is dominated by a few catastrophic pixels – which is why salt-and-pepper at $p = 0.05$ scores MSE 0.02416 while Gaussian $\sigma = 0.1$ scores 0.00541. MAE weights all errors linearly, and reverses that ranking (0.02508 versus 0.04396). SSIM ignores absolute pixel values and compares local means, variances and covariances, so it penalises blur that MSE tolerates – the FC autoencoder looks acceptable at MSE 0.021 and mediocre at SSIM 0.752. Reporting one alone would have supported the wrong conclusion at least once here.
13. **What is a Variational Autoencoder?** A generative model that learns a probability distribution over the latent space rather than a point per input. The encoder outputs $\mu(x)$ and $\log \sigma^2(x)$ defining $q_\phi(z|x) = \mathcal{N}(\mu, \text{diag}(\sigma^2))$; z is sampled from it and decoded. Training maximises the evidence lower bound (ELBO): reconstruction accuracy minus the KL divergence from the prior.
14. **How does a VAE differ from a conventional autoencoder?** Three ways. (i) The code is a distribution, not a point. (ii) The loss carries a KL regularisation term in addition to reconstruction. (iii) Sampling makes it generative – a plain autoencoder’s decoder, fed a random vector, generally produces nothing meaningful, because nothing forced its code space to be populated everywhere. Empirically the VAE reconstructed slightly *better* than the deterministic AE at the same $d_z = 2$ (SSIM 0.4486 versus 0.4386) while also being able to generate.
15. **Why does a VAE learn a distribution instead of a single latent vector?** To make the latent space continuous and complete. Every training image claims a small *region* rather than a point, those regions are pushed to overlap and to cover the prior, and consequently unvisited points still decode to something sensible. That is what makes sampling and interpolation work. The distribution also expresses uncertainty: an ambiguous digit can be encoded with a larger σ .
16. **What is the reparameterization trick?** Writing $z = \mu + \sigma \odot \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$ drawn independently of the parameters. Sampling is a non-differentiable operation, so gradients cannot pass through $z \sim \mathcal{N}(\mu, \sigma^2)$ directly. This reformulation makes z a deterministic, differentiable function of μ and σ with the randomness supplied externally, giving $\partial z / \partial \mu = 1$ and $\partial z / \partial \sigma = \epsilon$, so the encoder trains by ordinary backpropagation.
17. **Why is ϵ sampled from a standard normal distribution?** Because the prior is $\mathcal{N}(0, I)$, and any Gaussian can be produced from a standard one by the affine map $\mu + \sigma \epsilon$ – so the trick gives exactly the intended posterior. A standard normal is also parameter-free (no gradient can flow into it, which is the point), cheap to sample, and it makes the KL term available in closed form, $-\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$, so no Monte-Carlo estimate of the KL is needed.
18. **What is the role of KL divergence in the VAE loss?** It measures how far each posterior $q_\phi(z|x)$ is from the prior and penalises the distance, keeping codes centred, appropriately scaled and overlapping. Without it, the encoder would push posteriors apart into isolated islands with arbitrarily small variances – i.e. it would become a plain autoencoder – and the space between codes would decode to nothing. The measured KL here was 6.13 nats, and Exercise 6 shows the effect of moving it: $\beta = 0.5$ raises KL to 7.13 and improves reconstruction, $\beta = 4$ lowers it to 3.45 and makes reconstructions worse.
19. **What is the prior distribution used in the VAE?** The isotropic standard normal $p(z) = \mathcal{N}(0, I)$. It is chosen because it is simple to sample, its KL against a diagonal Gaussian posterior has a closed form, and it has no preferred direction, so it imposes no arbitrary structure. Its weakness showed up here: the aggregate posterior drifted away from it (mean(μ) = 0.295, digit 1 at $z_1 = 2.43$), and prior samples consequently over-represent whichever class happens to sit near the origin.

20. **Why is a two-dimensional latent space useful for visualization?** Because it can be plotted directly, with no t-SNE or PCA in between, so nothing about the geometry is an artefact of a projection. Plot 6 is literally the model’s internal representation. The price is reconstruction quality – SSIM 0.449 against 0.974 for the convolutional autoencoder – so $d_z = 2$ is a diagnostic setting, not a deployment one.
21. **What does latent-space interpolation demonstrate?** That the latent space is continuous and semantically organised rather than a lookup table. Decoding points along a line between two codes produced a gradual, plausible morph from 0 to 1 with no discontinuity and no ghostly double-exposure. It also exposed a limitation: consecutive steps differed by between 0.00089 and 0.01180 in pixel MSE, a ratio of 13.24, so equal steps in z do not mean equal visual change.
22. **Why can a VAE generate new images?** Because training aligns the aggregate distribution of codes with a prior we can sample from. Draw $z \sim \mathcal{N}(0, I)$, decode, and you obtain an image the model considers plausible but which does not exist in the dataset. A plain autoencoder cannot do this: nothing constrained where its codes live, so a random vector usually falls in a region the decoder never learned.
23. **Why might some VAE-generated images be unclear?** Four reasons, all visible in this run. (i) A 2-D latent cannot separate ten classes, so samples land between classes and decode to hybrids. (ii) Holes: with $\text{mean}(\sigma) = 0.071$ against $\text{std}(\mu) = 1.016$, the posteriors are near-points and large areas were never supervised. (iii) Prior–posterior mismatch: the codes are not centred on the origin, so the prior samples regions the data does not occupy. (iv) Pixel-wise likelihood: when the decoder is uncertain it minimises expected error by outputting a blurred average – hence 39% more ink in the generated samples than in the real ones.
24. **What is the difference between reconstruction and generation?** Reconstruction starts from a real image, encodes it and decodes it back; the target exists and can be compared pixel by pixel (MSE, MAE, SSIM). Generation starts from a latent vector with no corresponding image, so there is no target and no per-image error – quality has to be judged by realism and diversity. Reconstruction tests whether information survives the bottleneck; generation tests whether the latent space is properly organised. A model can be excellent at one and useless at the other, which is exactly the CAE/VAE contrast here.
25. **Why must test images remain unseen during training?** Because an autoencoder’s objective is to reproduce its input, so it is particularly easy to mistake memorisation for generalisation. If test images were in the training set, low test error would prove only that the model can reproduce images it has already fitted. The reported metrics – CAE MSE 0.00241, SSIM 0.97404 – are meaningful precisely because those 2,000 images influenced no weight and no design decision. The validation split existed so that architecture and convergence choices could be made without touching the test set.

29. Additional Exercises

Exercise 1: convolutional autoencoder with latent dimensions 4, 8, 16, 32

An explicit dense bottleneck was inserted between the convolutional encoder and decoder, giving the like-for-like comparison promised in Section 11.

d_z	MSE	MAE	SSIM	Parameters	Time (s)	FC MSE	FC SSIM
4	0.03491	0.08328	0.62910	102,725	21.24	–	–
8	0.02126	0.05525	0.76205	127,817	21.30	0.03040	0.65050
16	0.01327	0.03923	0.85307	178,001	21.18	0.02100	0.75181
32	0.00681	0.02534	0.92537	278,369	22.06	0.01997	0.75964

Observations. At every matched latent size the convolutional model beats the fully connected one: 37% lower MSE at $d_z = 16$, 66% lower at $d_z = 32$. The advantage *widens* with d_z , because the FC model saturates at $\text{SSIM} \approx 0.76$ while the convolutional one is still improving steeply at 32 – the FC decoder, not the code width, is the binding constraint. This confirms that Section 11’s headline result is genuinely architectural and not merely a consequence of the CAE’s 3136-value feature map.

Exercises 2 and 3: noise type and noise level in training

Three denoisers with identical architecture were trained on different corruptions and cross-evaluated. Mean SSIM (higher is better):

Trained on ↓ / Tested on →	Gauss. 0.1	Gauss. 0.2	Gauss. 0.3	S&P 0.10	S&P 0.20
Gaussian $\sigma = 0.1$	0.9655	0.8705	0.6784	0.7695	0.6379
Gaussian $\sigma = 0.2$	0.9601	0.9455	0.8709	0.8769	0.7399
Salt-and-pepper $p = 0.10$	0.8257	0.6523	0.5753	0.9461	0.8951

Test MSE (lower is better):

Trained on ↓ / Tested on →	Gauss. 0.1	Gauss. 0.2	Gauss. 0.3	S&P 0.10	S&P 0.20
Gaussian $\sigma = 0.1$	0.00303	0.00509	0.01058	0.00856	0.01665
Gaussian $\sigma = 0.2$	0.00366	0.00459	0.00696	0.00672	0.01287
Salt-and-pepper $p = 0.10$	0.00898	0.01592	0.02413	0.00478	0.00767

Salt-and-pepper denoising: matched vs mismatched training noise

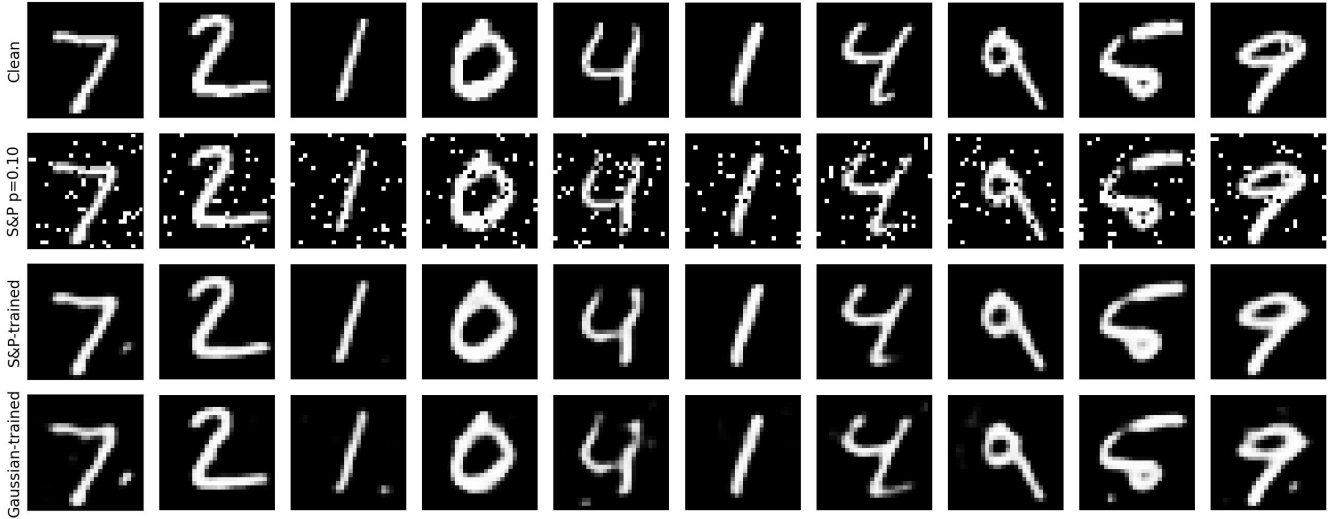


Figure 13: Salt-and-pepper denoising at $p = 0.10$: the matched (S&P-trained) model versus the mismatched (Gaussian-trained) model.

Observations.

- **Matched training always wins on its own noise** – the diagonal dominates every column. The $\sigma = 0.1$ model is best at $\sigma = 0.1$ (SSIM 0.9655 versus 0.9601), and the S&P model is best on S&P.
- **But matching narrowly is fragile.** The $\sigma = 0.1$ model collapses to 0.6784 at $\sigma = 0.3$, while the $\sigma = 0.2$ model holds 0.8709 there and gives up only 0.005 SSIM at $\sigma = 0.1$. Training slightly above the expected noise level costs almost nothing and buys a great deal of robustness.
- **Transfer is asymmetric.** The Gaussian-trained model is respectable on salt-and-pepper (0.8769 at $p = 0.10$ against the matched 0.9461), but the S&P-trained model is poor on Gaussian (0.6523 at $\sigma = 0.2$ against the matched 0.9455). Dense low-amplitude noise teaches general smoothing; sparse extreme-value noise teaches outlier rejection, which does not transfer. The figure shows the residue: the Gaussian-trained model leaves faint grey specks where the salt pixels were, because it was never trained to fully suppress isolated saturated pixels.

Exercise 4: transposed convolution versus upsampling in the decoder

Decoder	MSE	MAE	SSIM	Parameters	Time (s)
UpSampling2D + Conv2D	0.00241	0.01464	0.97404	74,497	23.13
Conv2DTranspose	0.00219	0.01378	0.97718	111,425	23.57

Decoder comparison: UpSampling2D vs Conv2DTranspose

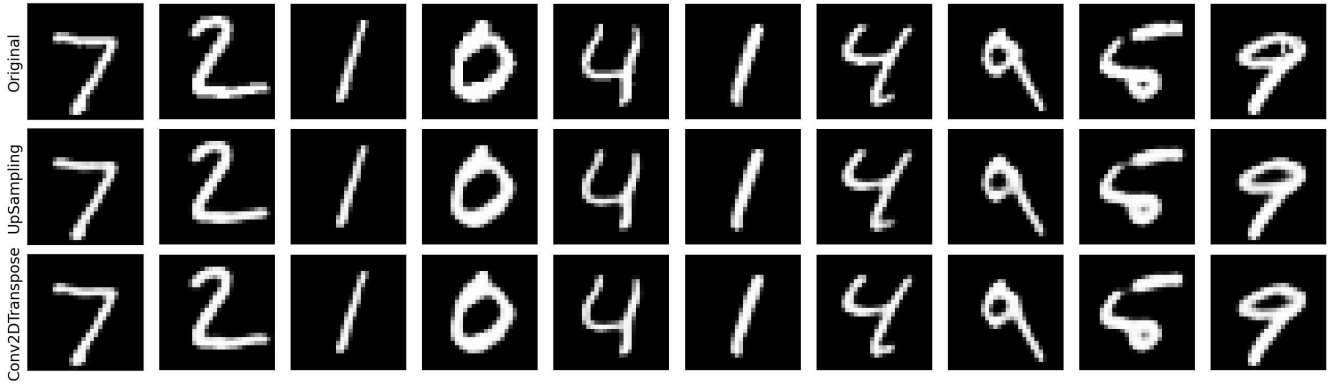


Figure 14: Decoder comparison: originals, UpSampling2D reconstructions, and Conv2DTranspose reconstructions.

The transposed-convolution decoder is marginally better (9% lower MSE, +0.003 SSIM) for 50% more parameters and the same training time. Visually the two rows are indistinguishable. Transposed convolution learns its upsampling kernel rather than using fixed nearest-neighbour duplication, which is why it edges ahead; at MNIST resolution the difference is not worth the parameters. No checkerboard artefacts appeared, which can occur with transposed convolutions when kernel size is not divisible by stride – here 3×3 with stride 2 is mildly susceptible, but the final 3×3 convolution smooths it away.

Exercises 5 and 6: latent dimension and KL weight in the VAE

d_z	β	MSE	MAE	SSIM	Rec loss	KL loss	Total loss	Active dims
2	1.0	0.0461	0.1078	0.4486	155.18	6.13	161.31	2 of 2
8	1.0	0.0466	0.1100	0.4458	158.55	6.23	164.78	4 of 8
2	0.5	0.0448	0.1029	0.4762	152.16	7.13	155.72	2 of 2
2	4.0	0.0460	0.1089	0.4414	157.61	3.45	171.40	2 of 2

(“Active dims” counts latent dimensions whose μ has standard deviation above 0.1 across the test set.)

Exercise 5 – latent dimension 2 versus 8. Giving the VAE four times more latent capacity produced *no* improvement: SSIM 0.4458 against 0.4486, reconstruction loss slightly worse. The reason is in the last column – only 4 of the 8 dimensions are active; the other 4 collapsed to the prior and carry no information. This is posterior collapse, and it is the characteristic failure mode of VAEs: a dimension that is not yet useful pays a KL cost, so the optimiser switches it off, and once off it receives no gradient to switch back on. A deterministic autoencoder has no such mechanism, which is why its quality rose monotonically with d_z in Section 27.

Exercise 6 – increasing the KL contribution. The effect is monotone and in the predicted direction. $\beta = 0.5$ relaxes the constraint: KL rises to 7.13 (codes spread further from the prior), reconstruction improves to 152.16 and SSIM to 0.4762. $\beta = 4$ tightens it: KL is squeezed to 3.45, the code carries less information, and reconstruction degrades to 157.61 with SSIM 0.4414. The trade-off is explicit – β buys prior-conformity, which generation needs, by spending reconstruction accuracy.

Exercises 7 and 8: a larger sample grid and centroid interpolation

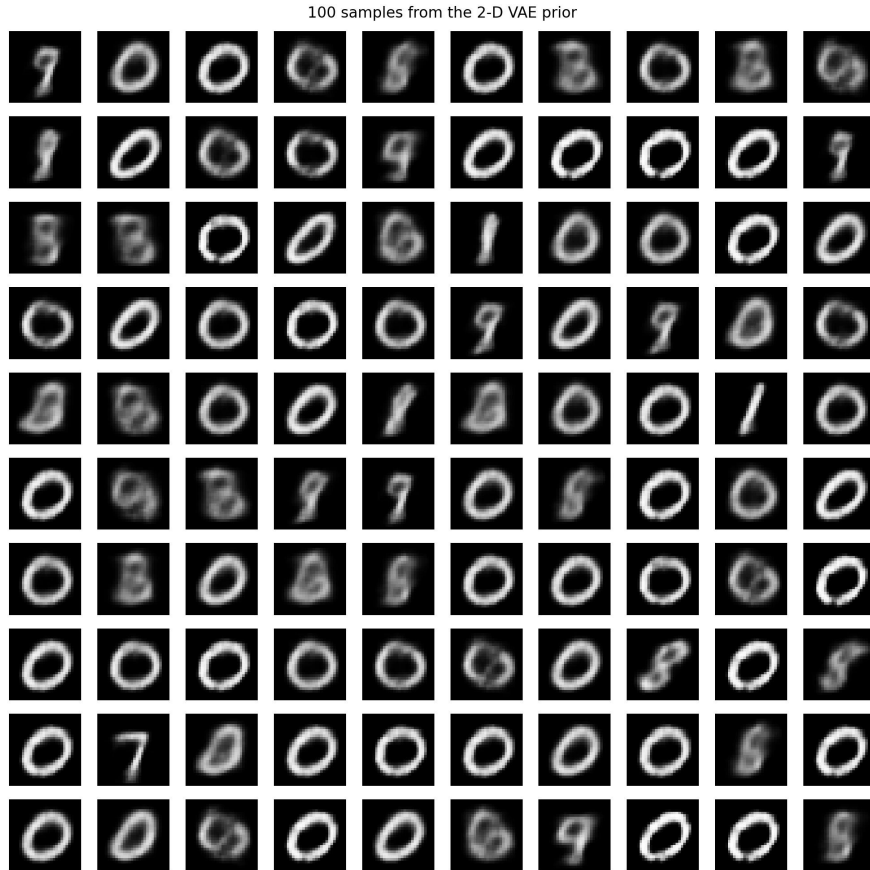


Figure 15: 100 samples drawn from the 2-D prior.

The larger grid confirms the diversity problem quantitatively rather than anecdotally: ring-like 0 shapes account for roughly 60% of the 100 samples, with the remainder mostly 8/9/3-like hybrids, a handful of 1s and one 7-like form. Digits 2, 4, 5 and 6 essentially never appear. This is the prior-posterior mismatch of Section 19 seen at scale: the prior places most of its mass near the origin, and the origin is 0 territory.

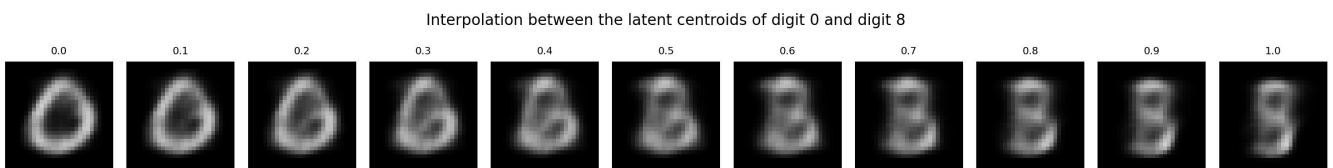


Figure 16: Interpolation between the latent centroid of digit 0, $(0.122, 0.045)$, and that of digit 8, $(0.854, -0.130)$.

Interpolating between class centroids rather than individual samples gives a cleaner morph, because both endpoints sit in dense, well-supported regions. The loop of the 0 progressively pinches inward, passes through 6-like forms around $\alpha = 0.3$ – 0.4 , and closes into a two-loop 8 by $\alpha = 0.7$. That the path between two class means stays on the digit manifold throughout is direct evidence that the KL term produced a connected latent space.

30. Expected Outcome

The complete image-representation and reconstruction pipeline was demonstrated:

Image $\rightarrow$ Encoder $\rightarrow$ Latent Representation $\rightarrow$ Decoder $\rightarrow$ Reconstructed Image

denoising:

Clean Image $\rightarrow$ Noise $\rightarrow$ Denoising Autoencoder $\rightarrow$ Recovered Image

and variational generation:

$$\text{Image} \rightarrow (\mu, \sigma) \rightarrow z \rightarrow \text{Decoder} \rightarrow \text{Generated/Reconstructed Image}$$

Overall conclusion. Architecture mattered more than capacity: the convolutional autoencoder cut MSE by 88.5% against the fully connected model while using 65% fewer parameters, and kept that advantage (37% lower MSE) at a matched 16-dimensional bottleneck. Latent width mattered up to a point and then stopped – SSIM rose $0.439 \rightarrow 0.752$ from $d_z = 2$ to 16, then only to 0.760 at 32. Denoising cost a little clean-image fidelity and bought a great deal of robustness, holding SSIM above 0.87 at noise levels where the raw input scored 0.51. The VAE reconstructed worst of the four models (SSIM 0.449), but the fair comparison is the deterministic autoencoder at the same $d_z = 2$, which scored 0.4386 – almost the entire gap is the two-dimensional bottleneck, not the probabilistic formulation, and in exchange the VAE was the only model able to generate new images and to interpolate meaningfully between them. Its real limitations were elsewhere: a 2-D space cannot separate ten digit classes, its aggregate posterior drifted off the prior, and at $d_z = 8$ half the latent dimensions collapsed. Every numerical value in this report came from the author’s own execution.

31. Source Code

The complete implementation is present in the following repository: <https://github.com/D-Rockie/Deep-Learning-Lab-2026.git>

32. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Diederik P. Kingma and Max Welling, “Auto-Encoding Variational Bayes,” ICLR, 2014.
3. Pascal Vincent et al., “Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion,” *Journal of Machine Learning Research*, 2010.
4. Irina Higgins et al., “beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework,” ICLR, 2017.
5. Zhou Wang et al., “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE Transactions on Image Processing*, 2004.
6. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, “MNIST Handwritten Digit Database.”
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>
9. scikit-image Documentation: <https://scikit-image.org>